

条款20 当 `std::shared_ptr` 可能悬空时, 使用 `std::weak_ptr`

①核心作用: 作 `std::shared_ptr` 的观察者

没有义务释放对象的指针

解决循环引用问题

②基础用法:

```
#include <memory>
```

```
auto sptr = std::make_shared<int>(10);
```

```
std::weak_ptr<int> wptr(sptr);
```

 必须且只能用 `sptr` 初始化

```
std::weak_ptr<int> wptr2 = sptr;
```

```
wptr.expired();
```

 查看对象是否存活

```
wptr.lock();
```

 将 `wptr` 升级为强引用

③核心特性 共享控制块

a. 弱引用, 不拥有所有权 不影响强引用计数和对象生命周期

b. 只观察, 不管理 只检测对象是否存活

c. 依赖 `shared_ptr` 存在 无法单独创建

d. 不保证对象存活, 只验证内存块被占用

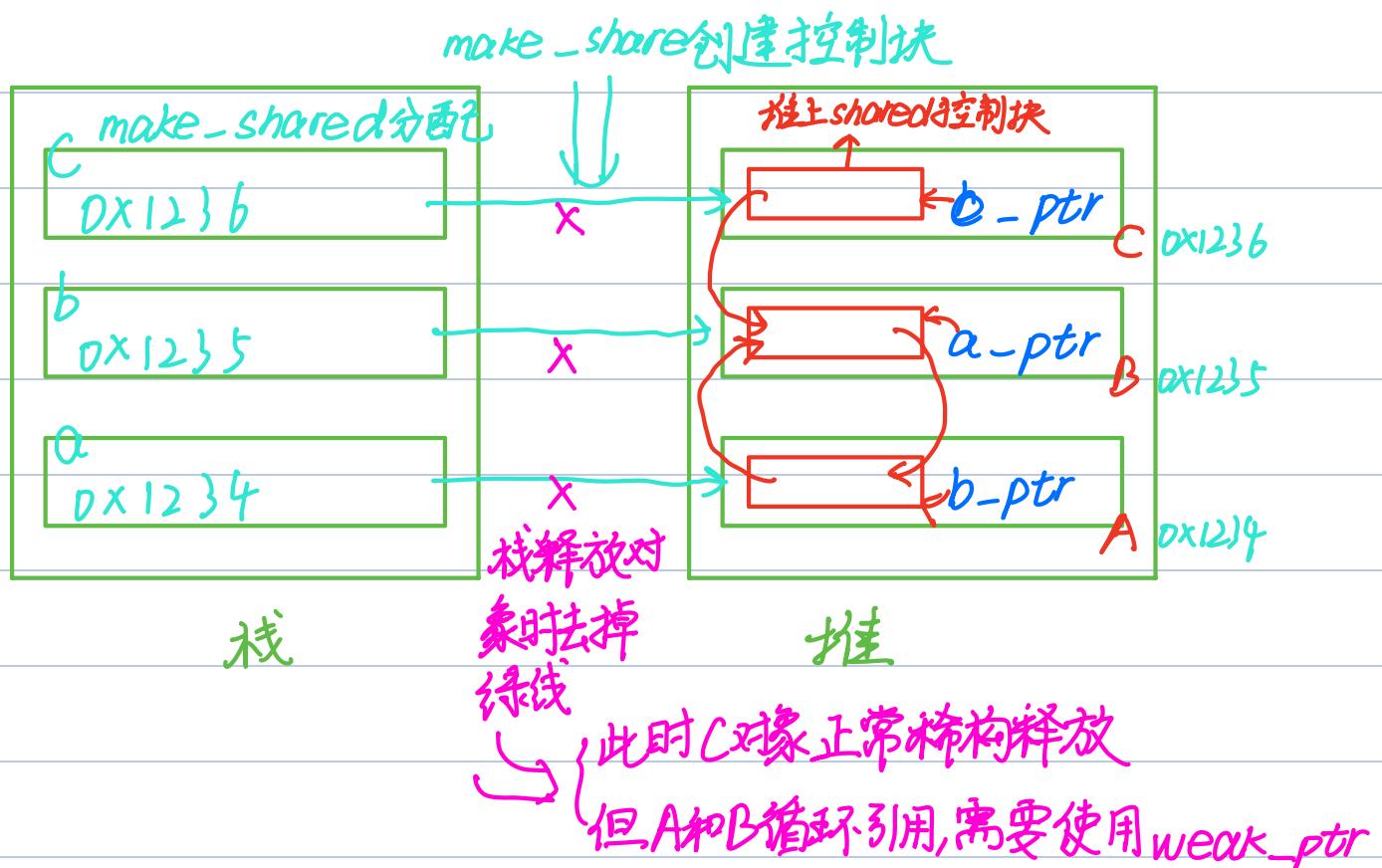
④经典使用, 查看对象的生命状态

⑤角色 监视者

```

class A { public: std::shared_ptr<B> b_ptr; }
class B { public: std::shared_ptr<A> a_ptr; }
class C { public: std::shared_ptr<B> b_ptr; }
auto b = std::make_shared<B>();
auto a = std::make_shared<A>();
auto C = std::make_shared<C>();
a->b_ptr = b; C->b_ptr = b; b->a_ptr = a;

```



## ⑥ lock() 升级为 shared 对象

核心作用: 返回临时的 shared\_ptr, 控制 T object 对象

a. 对象存活 → 返回有效 shared\_ptr 引用计数 + 1

b. 对象已销毁 → 返回空 shared\_ptr nullptr

原子操作: 查看强引用计数

相反 `expired()` 在线程内并不安全